

Sentiment Analysis with Transformers

Author: *Seyed Hossein Masoudi*

Supervisor: *Dr. Soheil Shamaee*

Department of Computer Science, Kashan

Introduction

This project focuses on Sentiment Analysis (Binary Classification) using a fine-tuned ParsBERT transformer model combined with a Multi-Layer Perceptron (MLP) classifier. The main objective is to automatically classify Persian user comments into positive or negative categories.

Dataset

- Source: Snapp comments dataset (originally used for collecting student feedback on instructor evaluations).
 - Language: Persian (Farsi).
 - Challenges: Raw data included noise, inconsistent spellings, stopwords, and special characters.
 - Preprocessing: Applied a custom JackageNormalizer function.
 - shape: (65973, 4)
 - columns: (comment, label, comment_length, comment_cleaned)¹
-

Hazm Library Issue

The well-known Hazm library (for Persian NLP) had compatibility issues with the TensorFlow environment:

- Hazm depends on numpy < 1.26.4, while TensorFlow requires newer versions.
- This conflict prevented smooth training and deployment.

Solution: A custom JackageNormalizer was implemented to replace Hazm. It includes 9 normalization steps:

1. Normalize Unicode

Normalizes the string into NFC form so that combined characters and diacritics are standardized.

2. Remove unwanted characters

Uses a regex to keep only ASCII characters, Persian block (U+0600–U+06FF), and spaces; everything else is removed like Emojis and symbols outside these ranges are removed.

¹ comment_cleaned is not used in the project.

3. Convert numbers

Converts digits between Persian ↔ English. With `to_english=True`, Persian digits “۰...۹” are converted to **0–9**.

4. Convert numbers to words

Converts numeric values into Persian words using `(num2fawords.words)` .

5. Standardize Persian text

Unifies Arabic characters to their Persian equivalents.

6. Remove keshide

Remove words that were dragged.

7. Remove punctuation

Removes punctuation marks and symbols.

8. Fix Persian ZWNJ

Fixes spacing and zero-width non-joiner (ZWNJ) issues. Normalizes multiple spaces. Corrects word+space + suffix patterns into proper ZWNJ form (“بهترین” ← “بهتر ین”, “کتاب ها”, “کتاب ها” ← “کتاب ها”).

9. Remove stopwords

Removes Persian stopwords. It becomes particularly important when dealing with large text corpora where computational efficiency matters. Processing every single word including high-frequency stopwords can consume unnecessary resources and potentially skew analysis results. Since some stop words files contain attributes such as good and bad, and these attributes have an important position in the analysis, they should be removed from the (Persian_StopList.txt) file.

Train Test split

The `train_test_split()` method is used to split our data into train and test sets. We need to divide our data into features (X) and labels (y). The dataframe gets divided into `X_train`, `X_test`, `y_train`, `y_test` and `y_val` .

Train set: The training dataset is a set of data that was utilized to fit the model. The dataset on which the model is trained. This data is seen and learned by the model.

Test set: The test dataset is a subset of the training dataset that is utilized to give an accurate evaluation of a final model fit.

validation set: A validation dataset is a sample of data from your model's training set that is used to estimate model performance while tuning the model's hyperparameters.

The data is divided into these categories:

- 70% → train
 - 15% → validation
 - 15% → test
-

Tokenizer

Since NLP models process numeric data, not text, there must be a translation between text and tokens. A token is an integer that represents a character or a short segment of characters. On the input side, the input text is parsed into a sequence of tokens. Similarly, on the output side, the output tokens are parsed into text. The module that performs the conversion between texts and token sequences is a tokenizer, and each model on the Hugging Face site has its own tokenizer, which we use to tokenize the ParsBERT model for our data.

The (`model_tokenizer`) converts each text into token IDs and generates the required attention masks. specifies that any text longer than the maximum length will be truncated with using (`truncation=True`) and with using (`padding=True`) ensures that all sequences are padded to the same length.

Model learning environment

The Google Colab environment and the T4 processor were used to train the model and set TensorFlow to use the legacy Keras implementation for compatibility.

Uses (`TF_USE_LEGACY_KERAS=1`) so TensorFlow routes Keras APIs to the legacy Keras package. This can avoid serialization/saving/loading mismatches between `tf.keras 3.x` and older code. And in the colab environment, two libraries `num2fawords` and `parsivar` which are for working in NLP are installed

Model Architecture

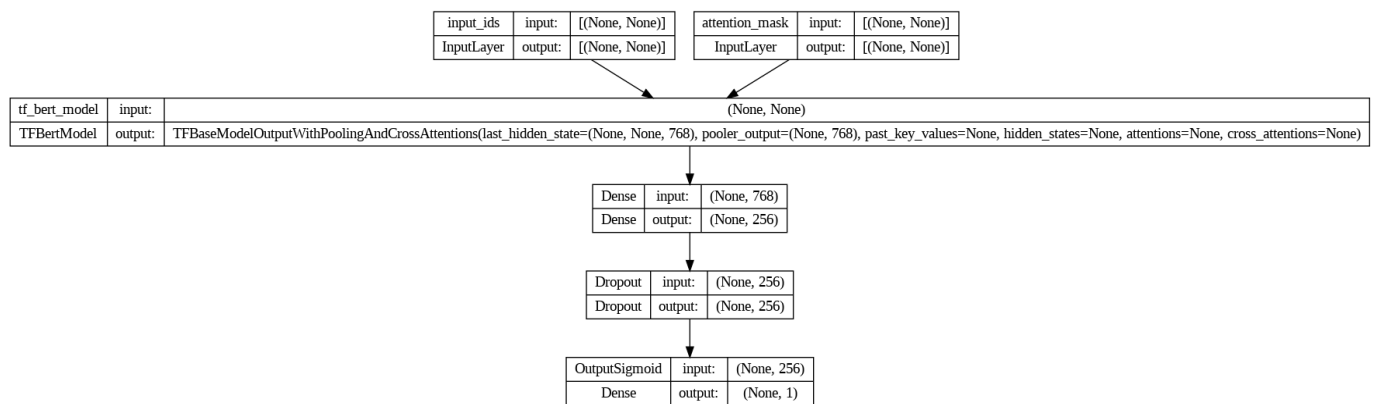
The architecture integrates ParsBERT embeddings with a neural classification head:

- Inputs: `input_ids`, `attention_mask`
- Base model: `TFBertModel (ParsBERT)`²
- Hidden layers:
 - Dense layer: 768 → 256 (GeLU activation)³
 - Dropout for regularization
 - Final Dense layer: 256 → 1 (Sigmoid activation for binary classification)

² [HooshvareLab/bert-base-parsbert-uncased](#)

³ I used the GLEU activation function because the activator of the Bert model itself was also gelu.

Model Diagram:



Stage 1: ⁴

Fine-Tuning (20 epochs) with transformers.trainable = False

Non-trainable params: 162841344 (621.19 MB)

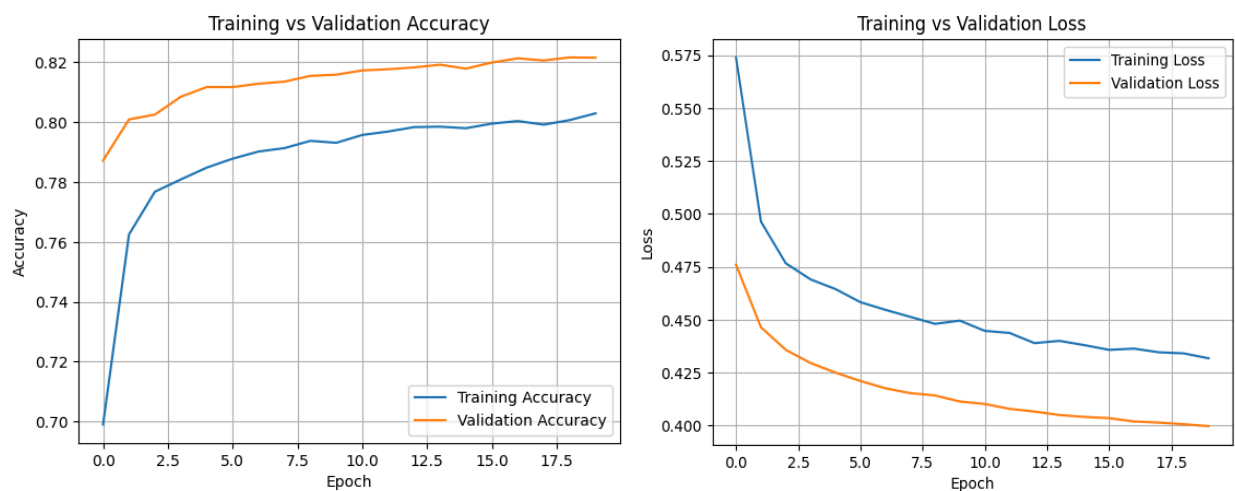
Trainable params: 197121 (769.50 KB)

- Optimizer: AdamW

- Learning rate: 1e-5

- Achieved about 80% validation accuracy by epoch 20, with AUC around 0.88.

The first Stage was implemented in such a way that to improve the learning of the model, we increase the parameters and place the model as a layer in a Feed for Forward network. With the help of model summary, it can be seen that 197121 Trainable parameters have been added to the model.



⁴ Commit ID: bbea3182a25fe79146957f236c48f21c7612d900

It can be seen from the model history that from around Epoch 10 onwards, the Train accuracy grows very slowly and becomes almost constant. This indicates that the model has reached a learning plateau and has no more capacity to extract patterns. The Train error decreases slowly and does not get very low, indicating that the model has not yet fully fit the Train data.

Stage 2:⁵

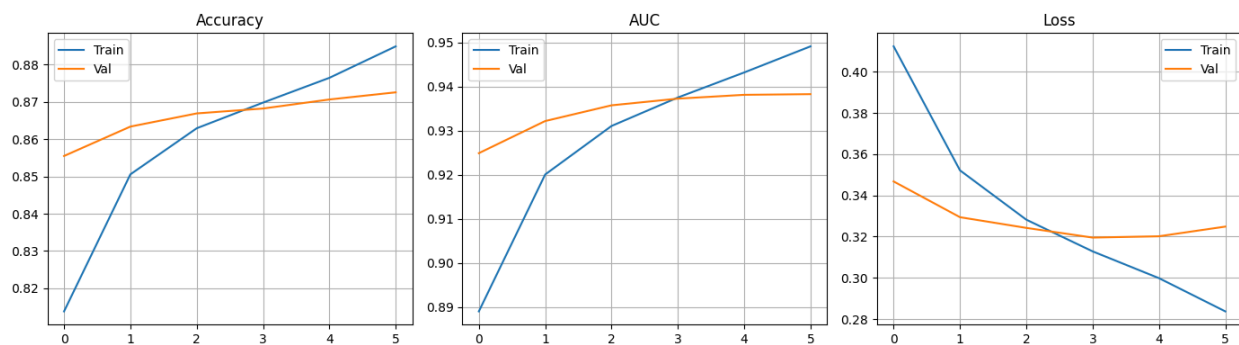
Fine-Tuning (5 epochs) with last 2 encoder layers unfrozen

Trainable params: 92159745 (351.56 MB)

Non-trainable params: 70878720 (270.38 MB)

- Reduced learning rate: 5e-6
- Optimizer: AdamW
- Achieved about 87% validation accuracy and AUC of 0.94.

To solve the problem of learning the model, we tried to increase the trainable parameters. It was possible to increase the parameters of the MLP network, but it would only increase the size of the model, so we opened the learning of the parameters of the last two layers of our parsBERT model.



The model is learning well (improving Train uniformly). It also has a good balance on Validation, without any sharp drops. A high and stable AUC is a sign of excellent quality in class recognition (the AUC values are above 0.92).

Key Observations:

- The model converged well in early epochs.
- Some overfitting was observed after epoch 3 in stage 2 but because of the early stopping, the model doesn't overfit much.
- Best balance between accuracy and generalization was achieved around epoch 3–4.

⁵ Commit ID : 5c44a74a0740030115fc2c318f3847a6d160d592

Callbacks

To optimize the training process and prevent problems such as overfitting or local model sticking, three important Callbacks are used in the TensorFlow/Keras environment:

1. Early Stopping

This mechanism is responsible for stopping training when the model no longer shows progress on the validation data. In this project, the value `patience=2` is considered. This means that if the validation error (`val_loss`) does not improve in two consecutive epochs (epoch), the training process ends. Also, the (`restore_best_weights=True`) option causes the best model weights to be restored. This prevents the model from overlearning on the training data.

2. Saving the best model (ModelCheckpoint)

In this project, a mechanism is implemented to save the best model weights based on the performance on the validation data. The `save_best_only=True` parameter ensures that only models that have significantly improved the validation error compared to previous times are saved. This way, at the end of training, the best version of the model will always be available in the specified file (here: "best_model_weights(stage2).h5").

3. Reducing the learning rate when progress stops (ReduceLROnPlateau)

One of the common problems in training neural networks is getting stuck in local minima or slowing down the convergence rate. To deal with this issue, the `ReduceLROnPlateau` mechanism is used. This callback is set to halve the learning rate (`factor=0.5`) if the validation error does not improve within a period (`patience=1`). In addition, the minimum learning rate is defined as `1e-6` so that the model does not stop moving.

User Interface (Streamlit and Gradio)

In the model load test section, the Gradio library was used to have a simple UI output. With the help of this library, the model can be output to the Gradio server for a reasonable time. But for better deployment of the model, a suitable UI has been created by using Streamlit, the parts of which are as follows:

1. The user enters a Persian text comment.
2. The text undergoes preprocessing: normalization and spell correction (via external API⁶).
3. The processed input is passed to the model.
4. Output: Sentiment prediction (Positive / Negative).

⁶ Explained in the Spell Correction section.

Spell Correction

Since user-generated text often contains spelling errors, spelling errors are corrected with the help of the [sapling⁷](https://sapling.ai/) site.

- An external Spell Checking API was integrated.
- If a typo is detected, the corrected word is substituted.
- If no error exists, the original word remains unchanged.

This step significantly improved the model's accuracy in noisy inputs and helped the model predict better.

Project Highlights

- Fine-tuned ParsBERT with a custom MLP head.
 - Replaced Hazm with a robust custom normalizer.
 - Achieved high performance on noisy Persian dataset.
 - Deployed a Streamlit UI for real-world testing.
-

Reference:

ParsBERT: Transformer-based Model for Persian Language Understanding
Authors: Mehrdad Farahani, Mohammad Gharachorloo, Marzieh Farahani, Mohammad Manthouri
ArXiv, 2020

The sentiment analysis project has been a valuable and challenging experience for me, marking an important step in my journey of learning and personal growth. All stages of the development and implementation are available in my GitHub repository, and I truly hope this work proves useful and rewarding for you.

Seyed Hossein Masoudi

⁷ <https://sapling.ai/>